

Hexlet AI - Проект №2 (Календарь - продолжение)

Обновлено 23 июня 2026, 15:31 | Михаил Иконников

Содержание страницы

Календарь звонков (продолжение)

[Чему учимся](#)[Описание проекта](#)[Что должно получиться](#)[0. Создание репозитория на GitHub](#)[1. Начало работы](#)[Чему учимся](#)[Описание проекта](#)[Что должно получиться](#)[Задачи](#)[Результат шага](#)[2. Установка и базовая настройка OpenCode в GitHub](#)[Ссылки](#)[Задачи](#)[Результат шага](#)[3. Работа с issue и triage](#)[Ссылки](#)[Задачи](#)[Результат шага](#)[4. От issue к pull request и ревью](#)[Ссылки](#)[Задачи](#)[Результат шага](#)[5. Регулярные задачи по расписанию](#)[Ссылки](#)[Задачи](#)[Результат шага](#)[6. Финальная проверка интеграции](#)[Задачи](#)[Результат шага](#)[7. Как мы проверяем проект](#)[Что должно работать](#)[Требования к оформлению](#)[Что дальше](#)

Календарь звонков (продолжение)

Проект модуля #2: Интеграция работы агентов в GitHub проект

Срок: 2-4 недели

Чему учимся

В этом проекте мы переносим агентный workflow из локальной разработки в GitHub-процесс команды.

Работаем не с пустым репозиторием, а с уже готовым приложением (созданным ранее).

Главная цель: встроить агентскую разработку в реальный цикл разработки на GitHub - от issue до pull request, ревью и регулярных автоматических проверок по расписанию.

Описание проекта

Базовый продукт уже должен быть реализован: сервис бронирования слотов по мотивам [Cal.com](#).

В этом продолжении мы не расширяем продуктивную функциональность, а добавляем организационную обвязку:

- Настройка OpenCode GitHub App.
- Работа с issue через комментарии.
- Создание и доработка PR агентом.
- Регулярные ночные проверки (например, Lighthouse) через schedule workflow.

Проект показывает, как использовать агента как участника командного процесса, а не только как локального помощника в IDE/CLI.

Что должно получиться

В репозитории должны появиться рабочие GitHub Actions и артефакты процесса:

- Ответ агента в issue.
- Пример triage/разбора задачи.
- PR с правками от агента и доработками после ревью.
- Scheduled workflow с отчетом по регулярной проверке.

0. Создание репозитория на GitHub

1. Подключение к GitHub

Код проекта будет храниться в репозитории на вашем компьютере и на GitHub. Хекслет автоматически создаст и настроит для вас репозиторий, для этого потребуются права на доступ к вашему GitHub-аккаунту. Хекслет будет управлять только репозиториями проектов курса. Ваши приватные репозитории и персональные данные полностью конфиденциальны.

2. Создание репозитория

Хекслет создаст репозиторий с названием [ai-for-developers-project-387](#) в вашем аккаунте на GitHub. Название репозитория важно для правильной работы автоматических проверок — не меняйте его. Если случайно удалите или отредактируете служебные файлы в директории `.github/workflows`, снова нажмите на кнопку «Подготовить репозиторий» для их восстановления. Написанный вами код, который хранится в других директориях, при восстановлении сохранится.

1. Начало работы

Это продолжение предыдущего проекта с календарем звонков. Само приложение уже реализовано.

В этом проекте мы настраиваем процесс командной работы с OpenCode в GitHub.

Задача не в том, чтобы написать новую фичу с нуля, а в том, чтобы встроить агента в существующий цикл разработки:

- issue;
- triage и анализ задачи;
- pull request;
- ревью и доработки;
- регулярные автоматические проверки.

Чему учимся

Главная цель: встроить агентскую разработку в реальный цикл разработки на GitHub - от issue до pull request, ревью и регулярных автоматических проверок по расписанию.

Описание проекта

Базовый продукт уже должен быть реализован: сервис бронирования слотов по мотивам [Cal.com](#).

В этом продолжении мы не расширяем продуктивную функциональность, а добавляем организационную обвязку:

- Настройка OpenCode GitHub App.
- Работа с issue через комментарии.
- Создание и доработка PR агентом.
- Регулярные ночные проверки (например, Lighthouse) через schedule workflow.

Проект показывает, как использовать агента как участника командного процесса, а не только как локального помощника в IDE/CLI.

Что должно получиться

В репозитории должны появиться рабочие GitHub Actions и артефакты процесса:

- Ответ агента в issue.
- Пример triage/разбора задачи.
- PR с правками от агента и доработками после ревью.
- Scheduled workflow с отчетом по регулярной проверке.

Задачи

1. Каждый проект на Хекслете выполняется в своем репозитории. Поэтому перенесите код из предыдущего проекта в новый репозиторий.
2. Определите план разработки: новые фичи, баги, которые будет исправлять агент.

Результат шага

- В репозитории лежит код приложения из предыдущего приложения.

- План по дальнейшему развитию приложения (фичи, баги).

2. Установка и базовая настройка OpenCode в GitHub

Подключаем OpenCode GitHub App к репозиторию и проверяем, что агент вызывается из комментария.

Ссылки

- [OpenCode GitHub Documentation](#)
- [OpenCode GitHub Installation](#)

Задачи

1. Выполните GitHub App с агентом в репозиторий и настройте его.
2. Создайте тестовый issue и вызовите агента через комментарий `/oc explain this issue`.

Результат шага

OpenCode подключен к репозиторию и подтвержден рабочий сценарий вызова агента из issue.

3. Работа с issue и triage

На этом шаге моделируем реальную командную ситуацию: в репозитории появляется неидеально описанная проблема, и агент помогает быстро разобрать ее.

Ссылки

- [OpenCode GitHub - Supported Events](#)

Задачи

1. Создайте issue с расплывчатым описанием проблемы в календарном приложении.
2. Добавьте комментарий с командой `/oc explain` и попросите агента разобрать проблему.
3. Если разбор полезный, попросите следующий шаг: предложить план исправления или сразу подготовить fix через `/oc fix`.
4. Сравните, насколько ответ агента помог уточнить задачу: причины, затронутые части кода, ожидаемый путь решения.

Результат шага

Issue прошел через triage с участием агента, и из нечеткого описания получилась рабочая техническая постановка.

4. От issue к pull request и ревью

На этом шаге отрабатываем полный путь: попросить агента создать или обновить PR, затем провести ревью и получить доработки в той же ветке.

Репозиторий перенесен из предыдущего проекта, где уже настроены Conventional Commits и release-please. Поэтому цикл не заканчивается на мёрдже PR: после слияния в основную ветку release-please сам формирует release-PR с changelog и предложенной версией. В агентной разработке это особенно важно — попросите агента писать коммиты по Conventional Commits (`feat:` , `fix:` и так далее), и тогда changelog с версией соберутся автоматически, без ручного труда.

Ссылки

- [OpenCode GitHub - Pull Request Example](#)
- [Conventional Commits](#) — спецификация формата сообщений коммитов.
- [release-please](#) — генерация release-PR с changelog и версией на основе Conventional Commits.

Задачи

1. В том же issue попросите агента подготовить PR с исправлением (если PR еще не создан).
2. Проверьте PR: описание, изменения, связь с issue.
3. Оставьте два вида замечаний:
 - общий комментарий в обсуждении PR;
 - комментарий к конкретной строке в diff.
4. Добавьте `/oc` в комментарии и попросите агента внести правки.
5. Проверьте, что агент обновил PR и учел оба типа обратной связи.
6. Убедитесь, что коммиты агента в PR оформлены по Conventional Commits (`feat:` , `fix:` и так далее).
7. Смерджите PR и проверьте, что release-please создал или обновил release-PR с changelog и предложенной версией.

Результат шага

Получен рабочий цикл " `issue -> PR -> review -> update -> merge` " с участием агента, а после слияния `release-please` автоматически формирует `release-PR` с changelog и версией по коммитам.

5. Регулярные задачи по расписанию

Добавим повторяющуюся задачу: она запускается по расписанию и при необходимости вручную.

Ночью агент выполняет проверку через `Lighthouse CLI`, а утром команда смотрит отчет и решает, нужны ли правки.

Ссылки

- [OpenCode GitHub - Schedule Example](#)
- [GitHub Actions - Events that trigger workflows](#)
- [Lighthouse CI - Getting started](#)

Задачи

1. Добавьте повторяющуюся задачу с запуском по расписанию.
2. Добавьте возможность запускать ту же задачу вручную.
3. Настройте запуск `Lighthouse CLI` и генерацию отчета.
4. Убедитесь, что отчет сохраняется и его можно посмотреть утром.
5. По итогам отчета фиксируйте, какие правки нужны в проекте.

Результат шага

В проекте работает регулярная проверка с утренним отчетом: команда получает понятный результат и принимает решения о правках.

6. Финальная проверка интеграции

На этом шаге мы не добавляем новые функции.

Наша цель — убедиться, что агентный процесс в GitHub работает устойчиво, безопасно и предсказуемо.

Задачи

1. Перепроверьте триггеры workflow:
 - Агент запускается только на целевые команды.
 - Нет запусков на каждый комментарий подряд.
 - Есть защита от самозапуска и реакции на сообщения ботов.
2. Перепроверьте `permissions`:
 - У каждого workflow только минимально необходимые права.
 - Нет избыточных `write`-разрешений.
3. Перепроверьте контроль расходов:
 - Разумная частота `schedule`.
 - Модель выбирается под задачу (где можно — более дешевая).
 - Понятно, где смотреть run-ы в GitHub Actions.
4. Убедитесь, что вы ранее поработали с агентом в GitHub:
 - Вызвали агента в `issue`.
 - Агент создал PR или выполнил ревью кода.
 - Хотя бы один запуск `scheduled workflow`.
5. Сделайте короткую самооценку эффективности работы с агентом:
 - Отметьте для себя, сколько задач решилось с первого прохода.
 - Выделите задачи, где потребовалось несколько итераций.
 - Хороший признак, если задач, решенных с первого раза, больше.

Результат шага

Агент в GitHub работает стабильно: workflow не зацикливаются, лишних запусков нет, основные сценарии (`issue`, `PR`, `schedule`) работают, и вы понимаете, где агент помогает с первого раза, а где нужны доработки.

7. Как мы проверяем проект

Формально проект проходит простую проверку: сборку и запуск приложения в контейнере.

В этом проекте важны не только файлы в репозитории, но и рабочий процесс вокруг `issue`, `pull request` и `scheduled workflow`.

Статус проверки можно посмотреть в репозитории GitHub.

Вывод проверки находится во вкладке *Actions* в workflow `hexlet-check`.

Подробнее об автоматической проверке можно почитать [здесь](#).

Что должно работать

В репозитории должны быть подтверждены основные сценарии GitHub-интеграции:

- Вызов агента из issue-комментария работает и оставляет ответ.
- Агент может выполнить triage issue с кратким разбором проблемы.
- По задаче создается или обновляется PR с правками.
- Ревью через комментарий к конкретной строке и общий комментарий приводят к доработкам.
- Ночной workflow (schedule) выполняется и публикует результат проверки.

Требования к оформлению

- В репозитории установлен и настроен OpenCode GitHub App.
- В `.github/workflows/` есть рабочий workflow для комментариев (`issue_comment` / `pull_request_review_comment`).
- В workflow корректно настроены `permissions` и secrets API-провайдера.
- В `.github/workflows/` есть `scheduled workflow` для регулярной проверки (например, `Lighthouse`).
- В проекте зафиксированы практики безопасности: фильтрация событий, защита от заикливания, контроль расходов.

Что дальше

После успешной проверки можно развивать процесс автоматизации:

- Добавить `workflow_dispatch` для ручного запуска ad-hoc задач.
 - Разделить workflow на отдельные сценарии: `triage`, `review`, `schedule`.
 - Подключить внешние данные (например, `Sentry`, `Search Console`) для регулярного анализа.
 - Вынести повторяющиеся промпы в `skills` и стандартизировать их для команды.
-